



Domain-specific meta-embedding with latent semantic structures

Qian Liu^{a,b}, Jie Lu^{a,*}, Guangquan Zhang^a, Tao Shen^a, Zhihan Zhang^c, Heyan Huang^b

^a AAIL, University of Technology Sydney, NSW, Australia

^b School of Computer Science, Beijing Institute of Technology, Beijing, China

^c School of Electronic Engineering and Computer Science, Peking University, Beijing, China

ARTICLE INFO

Article history:

Received 20 February 2020

Received in revised form 29 September 2020

Accepted 16 October 2020

Available online 23 October 2020

Keywords:

Natural language processing

Semantic representation

Meta-embedding

Graph neural network

ABSTRACT

Meta-embedding aims at assembling pre-trained embeddings from various sources and producing more expressively powerful word representations. Many natural language processing (NLP) tasks in a specific domain benefit from meta-embedding, especially when the task suffers from low resources. This paper proposes an unsupervised meta-embedding method that jointly models background knowledge from the source embeddings and domain-specific knowledge from the task domain. Specifically, embeddings from multiple sources for a word are dynamically aggregated to a single meta-embedding by a differentiable attention module. The embeddings derived from pre-training on a large-scale corpus provide complete background knowledge of word usage. Then, the meta-embedding is further enriched by exploring domain-specific knowledge from each task domain in two ways. First, contextual information in the raw corpus is considered to capture the semantics of words. Second, a graph representing domain-specific semantic structures is extracted from the raw corpus to highlight the relationships between salient words, then the graph is modeled by a powerful graph convolution network to effectively capture rich semantic structures among words in the task domain. Experiments conducted on two tasks, i.e., text classification and relation extraction, show that our model outputs more accurate word meta-embeddings for the task domain, compared to other state-of-the-art competitors. .

© 2020 Elsevier Inc. All rights reserved.

1. Introduction

Word representation aims to capture the semantics of words based on their distributional properties in a large volume of natural language data and represents words in a computer-processed format such as fixed-length vectors [1,2]. Recently, several distributed word representation methods have been developed to capture the meaning of words. The methods represent each word as a vector in a real-valued, low-dimensional space, in a way that embeds the semantics and syntactics of words using neural networks [1], or dimensional reduction on the word co-occurrence matrix [2]. Some pre-trained language models have been devised on large-scale unlabeled text corpora, with a powerful ability to capture the meaning of words from the discourse context, such as BERT [3] and XLNet [4]. These methods have been widely used as the fundamental input representation of words in neural network-based models, and have been empirically proven to boost the performance in natural language processing (NLP) tasks such as text classification [5] and machine reading comprehension [6].

* Corresponding author.

Recent studies [7,8] have observed significant variance in the quality and characteristics of the semantics captured by word representations generated by various existing methods according to the type of the training corpus, how the learning architecture is designed, and whether or not external knowledge is considered. These observations imply that different methods capture varied and complementary aspects of lexical semantics [9]. Moreover, a pioneering study from Yin and Schütze [10] found that a combination of word representations from multiple methods leads to a better performance compared with an arbitrary individual method. They showed that it is feasible to combine the strengths of different word representations and thus yield a new representation space with improved overall expression quality. These novel observations address the importance of meta-embedding, which assembles different existing pre-trained word representations to yield a new and more powerful one.

Typically, meta-embeddings are derived from a set of source embeddings that are freely-available and pre-trained on large-scale corpora. To obtain meta-embeddings, most existing methods carry out straightforward mathematical operations over the set of source embeddings, such as concatenation [10], averaging [11], or constructing a new common embedding space by capturing complementary information in different source embeddings [12]. However, these methods treat different source embeddings with various qualities equally, and combine source embedding directly rather than adapting to the need of target tasks. To alleviate this problem, we propose to dynamically aggregate source embeddings with the attention schema to learn meta-embeddings.

Moreover, previous works did not consider the importance of knowledge from the target domain for meta-embeddings, which is crucial to downstream tasks. Therefore, there are two main problems in the learned meta-embeddings. Firstly, a semantic shift exists in the task domain and learning representation space. Naturally, the semantics of words vary across domains. For example, the semantics of *season* trained in a general domain might be closely related to *spring*, *summer*, *autumn*, and *winter*, but fail to be associated with a food flavoring which is specific to the kitchen domain. To alleviate the semantic shifts problem, we propose to explore the contextual information of words in the task domain, so as to guide the meta-embedding learning process. Secondly, the salient and specific words in the task domain are not well-characterized in the learned representation space. In natural languages, the distributions of salient and domain-specific words often determine the meaning of a document [13], but they are rarely appeared in other domains. As such, source domains often lack sufficient information for training high-quality embeddings for specific words. It is worth noting that using contextual information alone does not capture the relationships of domain-specific words well, especially in low-resource scenarios. For example, consider the sentence *I own many KitchenAid appliances and love this brand*. Even though the context of *KitchenAid* can effectively capture its feature as a *brand*, it will lose some strong indicators with other specific words in the kitchen domain, such as *mixer* and *blender*. To alleviate this problem in our method, we exploit the relationship between salient and specific words in the task domain. More specifically, we construct a graph of salient words and explicitly organize their correlations, then employ graph convolution networks (GCNs) on the graph to transfer the latent structures into the learned meta-embedding space.

In brief, in this work, we propose an unsupervised and domain-specific meta-embedding approach, which goes a step further than previous ensemble methods. It dynamically leverages different source embeddings as background knowledge in an attention-driven strategy and mines specialized features of the task domain as domain-specific knowledge. To capture specialized knowledge in the task domain, we explore two ways to integrate meta-embeddings with domain-specific knowledge: 1) The contextual information discovered from the raw corpus, and 2) The domain-specific semantic structures conveyed by a graph built on salient words, where a stack of GCNs is applied over for in-depth knowledge mining.

The contributions of this paper are summarized as follows.

- We develop an unsupervised, end-to-end trainable framework that addresses domain-specific meta-embedding by comprehensively exploring the task domain and various source domains. Instead of mining semantics from a single domain, our method is able to leverage knowledge from multiple domains, to generate high-quality and accurate word representations.
- We dynamically leverage different source embeddings with an unsupervised attention mechanism. Unlike the previous method, we provide an effective solution to dynamically combine multiples source embeddings with the attention schema, rather than using them equally.
- We explore latent semantics in the task domain in the meta-embedding learning process. Specifically, we apply GCNs over a domain-specific graph, which efficiently represents the significance of domain-specific words and their global correlations. As such, our method alleviates the problem of inadequate training on domain-specific words.

The rest of the paper is arranged as follows. We provide a brief overview of the related work in Section 2, and elaborate our meta-embedding method in Section 3. Then, in Section 4, we evaluate our method extensively on two well-known tasks, i.e., text classification and relation extraction. The experiment results consistently demonstrate the promising performance of our method. Furthermore, we offer more insightful analyses of the proposed method and assess the effect of each essential component in Section 5, and concluding remarks are provided in Section 6.

2. Related works

We begin by providing a brief outline of word representations, followed by an overview of the representative works of meta-embedding learning methods and the application of GCNs in the NLP domain.

2.1. Word representation

Word representation is the feature learning techniques in NLP where words are mapped to vectors of real numbers. In the early stages, words are represented by the one-hot encoding method, i.e., each word in the vocabulary is mapped into a high-dimension and sparse vector where only one dimension is set to a value of one. This one-hot method suffers from the dimensional curse and cannot convey the semantics and syntactics of words. To alleviate these problems, the distributed representation methods are proposed to represent words in a low-dimension and dense vector and each dimension captures the latent semantics of words.

Typically, distributed representation methods learn the semantics of words from unlabeled corpora following the distributed hypothesis, which suggests words with similar context have similar meanings. At first, a latent semantic analysis method is proposed to reduce the number of dimensions using singular value decomposition. The co-occurrence information between a target word and its context is generated by a sliding window. For example, considering the sentence “*the used seasonings include basil*”, the central word *seasonings* is regarded as a target word, and its surrounding words as context words. The semantics of words are unsupervised learned by predicting the target words using its context words (such as NNLM [14] and Word2Vec [1]) or decomposing the co-occurrence matrix of the target word and its context (such as GloVe [2]). Hereafter, word representation methods have been extensively studied based on neural networks [15], explicit representation in terms of the context in which words appear [16], and explainable knowledge base methods [17]. Moreover, several high-quality pre-trained word representations have been released. They are widely used as a basic block in neural network-based models and largely benefit various NLP tasks. For example, GloVe is trained on aggregated global word-word co-occurrence statistics from a corpus of over 840B tokens, and fastText [18] pre-trained word representations for 157 languages on Common Crawl and The Wikipedia Corpora.

Recently, several powerful contextual representation methods have been devised, which are also known as pre-trained language models [19]. They first pre-train deep language models on large-scale unlabeled text corpora, then fine-tune the models on downstream tasks, which are beneficial for downstream tasks and can avoid training models from scratch. Several high-quality pre-trained language models have been released, including BERT [3], and XLNet [4]. With a powerful ability to capture the meaning of words from the discourse context, they have led to significant performance gains and achieve state-of-the-art performance in various tasks, such as relation extraction [20] and machine reading comprehension [21]. Different from our method, pre-trained language models provide the contextual encoders for downstream tasks, which are linguistic knowledge and world knowledge. In this paper, we learn domain-specific representations for the specific task defined in the task domain, highlighting the importance of domain-specific knowledge.

2.2. Meta-embedding learning

With the design of various word representation methods, researchers have found that different methods vary significantly in the quality and characteristics of word semantics. Inspired by transfer learning methods [22], several works have shown that incorporating multiple word embeddings learnt from different public corpora can improve performance in various NLP tasks. For example, Tsuboi [9] showed that using Word2Vec [1] and GloVe [2] embeddings together could improve the tagging accuracy. As such, meta-embeddings have become a popular trend, which combine existing high-quality pre-trained word embeddings to produce more accurate and complete versions.

In an early attempt, Yin and Schütze [10] learned a projection layer that projects a word's meta-embedding to its source embeddings. Later, Coates and Bollegala [11] showed that the arithmetic mean of distinct source embeddings can yield meta-embeddings of a higher quality that are even better than the results from more complex meta-embedding learning methods. Bao and Bollegala [12] learned an auto-encoder as the meta-embedding space by accurately reconstructing all source embeddings simultaneously. Rather than reconstructing source embeddings, Bollegala et al. [23] proposed a locally linear meta-embedding method to reconstruct neighboring words in each source embedding space.

More recent ensemble methods have been designed to dynamically adapt source embeddings to task domains [24,25]. For instance, Kiela et al. [26] explored the supervised learning of task-specific meta-embeddings, and applied their technique to sentence representations. Xu et al. [27] learned domain-specific meta-embedding as a lifelong learning framework, which extracts the context of words from past domains, to enrich the in-domain corpus and alleviate data scarcity problems. Hazem and Morin [28] explored a variety of embedding models and investigated their impact on the task of bilingual terminology extraction from specialized, comparable corpora. They showed that meta-embedding, based on specialized and general domain datasets, can improve performance when mining specialized bilingual lexicons. These methods typically align the source and target embeddings in a common space under the guidance of pivots, which are shared across domains.

2.3. Graph convolution networks in NLP

Graph convolution networks [29,30] aim to extend standard convolution operations for general graph structures. GCNs have enjoyed wide success in the areas of computer vision [31,32], demonstrating their general applicability and strong learning power for capturing structural information [33]. Only a few methods explore the applications of GCNs for NLP tasks such as machine translation [34] and event detection [35]. Graph embeddings are also exploited for capturing semantics. For example, Liu et al. [36] proposed the use of graph-based semantic structures in knowledge bases under the assumptions of concept convergence and word divergence. Nguyen et al. [37] proposed learning for short-text semantic similarity with word embeddings and external graph-based knowledge sources.

In our method, GCNs are employed for learning domain-specific meta-embeddings. Here, semantic structures are captured by the graph of domain-specific words, where the nodes convey word semantics and the edges capture the correlations between words. GCNs are used to mine the latent structures of the graph, to capture domain-specific knowledge in the learned meta-embedding space.

3. Method

Given a set of pre-trained source embeddings, our goal is to learn meta-embeddings for a task domain. A set of N source embeddings is denoted as $\{\mathbf{E}^n \in \mathbb{R}^{|\mathcal{V}_n| \times d_n}; 1 \leq n \leq N\}$, where $\mathcal{V}_n$ and d_n are the vocabulary and dimension, respectively. The output meta-embeddings are denoted as $\mathbf{U} \in \mathbb{R}^{|\mathcal{V}| \times d}$, where $\mathcal{V}$ is the vocabulary of the task domain and d is the embedding dimension. The overall framework of our method is illustrated in Fig. 1. In this section, we first introduce how to integrate pre-trained source embeddings and generate meta-embeddings. Then, we detail how to guide the learning process to learn domain-specific meta-embeddings, considering both the contextual information and semantic structures. The used symbols and their descriptions are listed in Table 1.

3.1. Dynamically combined meta-embeddings

Given sufficient background knowledge of its context, humans can understand an unknown word. Inspired by this intuitive observation, we regard the pre-trained source embeddings as the background knowledge of words. To be specific, for each word w_i in the task domain, we first project its corresponding source embeddings $\{\mathbf{e}_i^n \in \mathbf{E}^n\}_{n=1}^N$ into a common d -dimensional space as follows:

$$\mathbf{s}_i^n = \mathbf{Z}^n \mathbf{e}_i^n + \mathbf{b}^n, \quad (1)$$

where $\mathbf{Z}^n \in \mathbb{R}^{d \times d_n}$ and $\mathbf{b}^n \in \mathbb{R}^d$ are the learnable linear projection parameters. If w_i is not in the source vocabulary $\mathcal{V}_n$, $\mathbf{s}_i^n$ is padded to the zero vector. We dynamically combine the projected source embeddings $\{\mathbf{s}_i^n \in \mathbb{R}^d\}_{n=1}^N$ as w_i 's context embedding $\tilde{\mathbf{x}}_i$ with the self-attention mechanism as follows:

$$\tilde{\mathbf{x}}_i = a_i^c \mathbf{c}_i + \sum_{n \in [1, N]} a_i^n \mathbf{s}_i^n, \quad (2)$$

where a_i^c is the scalar combination weight computed by an attention layer. A randomly initiated vector $\mathbf{c}_i$ is introduced to capture the latent features when w_i works as a contextual word for other words in the task domain.

Then, we represent the specific semantics of w_i in the task domain using $\mathbf{x}_i \in \mathbb{R}^d$. The final meta-embedding $\mathbf{u}_i$ of w_i is computed as the dynamic combination of $\tilde{\mathbf{x}}_i$ and $\mathbf{x}_i$ as

$$\mathbf{u}_i = a_i^t \mathbf{x}_i + \tilde{\mathbf{x}}_i = a_i^t \mathbf{x}_i + a_i^c \mathbf{c}_i + \sum_{n \in [1, N]} a_i^n \mathbf{s}_i^n, \quad (3)$$

where a_i^t is a scalar weight computed through a self-attention mechanism. More specifically, together with weights in Eq. (2), we compute the combination weights with an attention layer applied with a softmax function:

$$[a_i^t, a_i^c, a_i^1, \dots, a_i^N] = \text{softmax}(\mathbf{w}^\top [\mathbf{x}_i, \mathbf{c}_i, \mathbf{s}_i^1, \dots, \mathbf{s}_i^N]), \quad (4)$$

where $\mathbf{w}$ is the learnable parameter, and the bias term is omitted for brevity.

It is notable that the underlying features of w_i in the task domain are conveyed by $\mathbf{c}_i$ and $\mathbf{x}_i$. Inspired by Pennington et al. [2], they are designed to represent the semantic characteristics of w_i as a context word and a target word, respectively. They are randomly initiated following a standard normal distribution and optimized in the learning process. Moreover, these two vectors can avoid the domain-specific words being represented by zero vectors when they are out-of-vocabulary (OOV) words in all source domains.

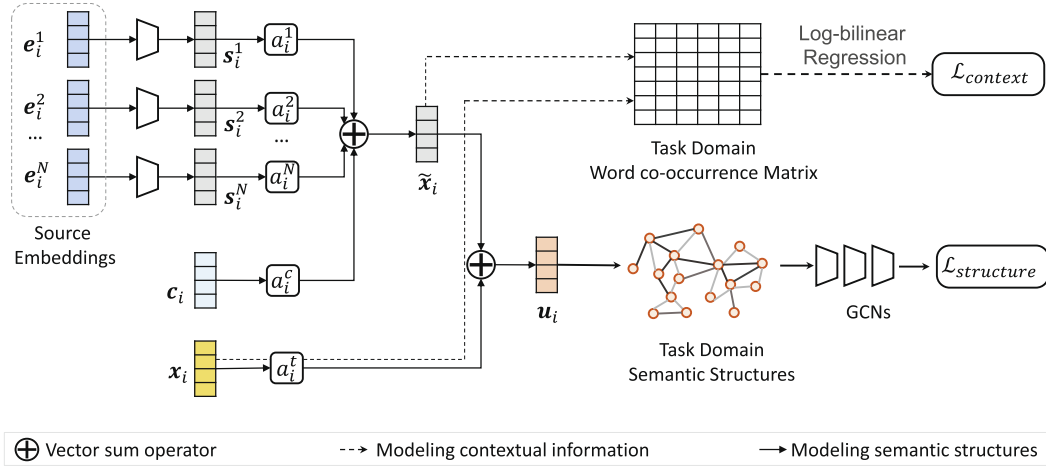


Fig. 1. An overview of the proposed meta-embedding framework. The dashed lines denote the workflow of modeling contextual information, and the solid lines denote the workflow of modeling semantic structures..

Table 1

Symbols and the descriptions. Matrix is denoted with bold capital letter, vector is denoted with bold lowercase letter, and scalar number is denoted with lowercase letter.

Symbol	Description	Symbol	Description
$\mathcal{C}$	Corpus.	$\mathcal{V}$	Vocabulary.
$\mathcal{G}$	Semantic graph.	$\mathcal{N}_i$	Neighbors of word w_i in $\mathcal{G}$.
$\mathbf{E}$	Source embedding matrix.	$\mathbf{e}_i$	Source embedding of word w_i .
d	Dimension of embedding.	$\mathbf{c}, \mathbf{x}$	The context embedding and the target embedding of word w_i .
$\mathbf{U}$	Meta-embedding matrix.	$\mathbf{u}_i$	Meta-embedding of word w_i .
$\mathbf{M}$	Co-occurrence matrix of $\mathcal{C}$.	m_{ij}	Element of $\mathbf{M}$, co-occurrence times of word w_i and w_j .
$\mathbf{P}$	Adjacency matrix of $\mathcal{G}$.	p_{ij}	Element of $\mathbf{P}$, edge weight of word w_i and w_j .
$\mathbf{A}$	Normalized symmetric adjacency matrix of $\mathcal{G}$.	$\mathbf{D}$	Degree matrix of $\mathcal{G}$.
$\mathbf{W}, \mathbf{w}, \mathbf{b}$	Trainable parameters.	a	Combination weight.

3.2. Domain-specific Knowledge

To capture the semantics of words in the task domain, we mine the domain-specific knowledge, then use this knowledge to guide the learning process of meta-embedding. Two kinds of knowledge with different granularities are considered, i.e., contextual information in the raw corpus and the underlying semantic structures.

3.2.1. Contextual information

The contextual information of words in a corpus is the primary source of information available to all unsupervised methods for learning word representations. Following Pennington et al. [2], contextual information in the task domain is organized into a co-occurrence matrix $\mathbf{M} \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|}$ using a sliding window on the task domain corpus $\mathcal{C}$, where the centre of the window is the target word and the other words are its context. The (i, j) -th element m_{ij} of $\mathbf{M}$ indicates the number of times word w_j appears in the context of word w_i . We use the log-bilinear regression model proposed by GloVe [2] to learn word representations, and the training objective is defined as:

$$\mathcal{L}_{\text{context}} = \frac{1}{2} \sum_i \sum_j f(m_{ij}) \left(\mathbf{x}_i^\top \tilde{\mathbf{x}}_j + b_i + \tilde{b}_j - \log(m_{ij}) \right)^2, \quad (5)$$

where $m_{ij} \in \mathbf{M}$, the central word is w_i and its contextual word is w_j , $\mathbf{x}_i$ is the defined specific representation of w_i , $\tilde{\mathbf{x}}_j$ is the context embedding of word w_j as defined in Eq. (2), and b_i and $\tilde{b}_j$ are real-valued scalar bias terms that compensate for the difference between the inner-product and the logarithm of the co-occurrence counts. The function $f(x)$ discounts the co-occurrences between frequent words and is given by:

$$f(m) = \begin{cases} (m/m_{\max})^\gamma & \text{if } m \leq m_{\max}, \\ 1 & \text{otherwise,} \end{cases} \quad (6)$$

where $m_{\max} = 100$ and $\gamma = 0.75$, as suggested in Pennington et al. [2].

3.2.2. Semantic structures

The context information is not sufficient to precisely and completely understand certain domain-specific words with specific meanings. This is mainly because the co-occurrence matrix $\mathbf{M}$ most likely considers all co-occurrences equally, which likely causes the domain-specific information deficiency problem, especially when the co-occurrence is rare. Thus, it fails to capture the rich relationships between words, especially for salient, domain-specific words. To alleviate this problem, we propose a GCN-based framework to further mine semantic structures among domain-specific words, which is essential for characterizing the task domain, and hence leads to more accurate meta-embeddings.

First, to identify domain-specific words, we adopt the term frequency-inverse document frequency (TF-IDF), which is an effective method for measuring the importance of words. The term frequency is the number of times a word appears in the document, and the inverse document frequency is the logarithmically scaled inverse fraction of the number of documents that contain the word. Then, we build an undirected graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ to capture domain-specific structures among salient words, where $\mathcal{V}$ is the node set of salient words and $\mathcal{E}$ is the set of edges to capture the relationships among salient words. Each edge connects two words, w_i and w_j , and the edge weight p_{ij} accounts for their relevance score r_{ij} and their individual importance scores, o_i, o_j , in the task domain:

$$p_{ij} = \begin{cases} r_{ij} \cdot (\frac{o_i + o_j}{2}) & \text{if } i \neq j, \\ \alpha \cdot o_i & \text{if } i = j, \end{cases} \quad (7)$$

where $\alpha = 1$ is a hyper-parameter, which will be quantitatively evaluated in Section 5. The relevance score r_{ij} of two words w_i and w_j is defined as their co-occurrence times in a sentence or document. The importance scores o_i, o_j are obtained through TF-IDF.

For an intuitive comparison, considering a salient word *KitchenAid* in the kitchen domain, Fig. 2 shows the most related words obtained from the co-occurrence matrix $\mathbf{M}$ (left), and the domain-specific graph $\mathcal{G}$ (right). As can be observed, $\mathbf{M}$ focuses more on local context (due to the use of a local sliding window), while $\mathcal{G}$ is able to capture global semantic relations among significant words.

To encode the structures of graph $\mathcal{G}$ in the learned meta-embedding space, we apply the effective and powerful GCN over $\mathcal{G}$ to mine valuable domain-specific knowledge. When adapting one single GCN layer [30], we have:

$$\mathbf{H} = \sigma(\mathbf{AUW}), \quad (8)$$

where $\mathbf{U} \in \mathbb{R}^{|\mathcal{V}| \times d}$ is the input embedding matrix, its element $\mathbf{u}_i$ is the input embedding of w_i which is computed using Eq. (3)¹, $\mathbf{H} \in \mathbb{R}^{|\mathcal{V}| \times d}$ is the output hidden representation of the nodes, $\mathbf{A} \in [0, 1]^{|\mathcal{V}| \times |\mathcal{V}|}$ is the normalized symmetric adjacency matrix of $\mathcal{G}$, $\mathbf{W} \in \mathbb{R}^{d \times d}$ is a learnable weight matrix, and $\sigma(\cdot)$ indicates a non-linear activation function (e.g., *ReLU*). Here, $\mathbf{A}$ is computed as:

$$\mathbf{A} = \mathbf{D}^{-\frac{1}{2}} \mathbf{P} \mathbf{D}^{-\frac{1}{2}}, \quad (9)$$

where $\mathbf{P}$ is the adjacency matrix of $\mathcal{G}$ with a self-loop. $\mathbf{D}$ is the degree matrix, i.e., $d_{ii} = \sum_j p_{ij}$, $p_{ij} \in \mathbf{P}$.

Eq. (8) can be represented in an alternative form:

$$\mathbf{h}_i = \sigma(\sum_{w_j \in \mathcal{N}_i} \mathbf{W} \mathbf{u}_j), \quad (10)$$

where $\mathcal{N}_i = \{w_j : p_{ij} > 0\}$ indicates neighbors of w_i . The bias term is omitted for brevity. Due to the self-loops (Eq. (7)), we have $w_i \in \mathcal{N}_i$ and the input feature $\mathbf{u}_i$ of w_i affects its induced representation $\mathbf{h}_i \in \mathbb{R}^d$. In addition, as GCN operates on the neighbors of a node, the updated node representation $\mathbf{h}_i$ efficiently captures semantic structures over $\mathcal{G}$.

One single GCN layer only encodes information about immediate neighbors. By stacking multiple GCN layers, higher-degree neighborhoods can be incorporated thus capturing more complex semantic structures/correlations:

$$\mathbf{h}_i^{k+1} = \sigma(\sum_{w_j \in \mathcal{N}_i} \mathbf{W}^{k+1} \mathbf{h}_j^k), \quad \mathbf{h}_i^0 = \mathbf{u}_i. \quad (11)$$

The training objective is designed to predict a target word w_i given its neighbors $\mathcal{N}_i$ over $\mathcal{G}$, i.e., to maximize the following energy:

$$\sum_{w_i \in \mathcal{V}} \log \Pr(w_i | \mathcal{N}_i). \quad (12)$$

The training loss is thus defined as:

$$\mathcal{L}_{\text{structure}} = - \sum_{w_i \in \mathcal{V}} \left(\mathbf{u}_i^\top \mathbf{h}_i^k - \log \sum_{w_j \in \mathcal{V}} \exp(\mathbf{u}_j^\top \mathbf{h}_i^k) \right), \quad (13)$$

¹ Noting that the $\mathbf{U}$ here is the meta-embedding matrix with random initial parameters.

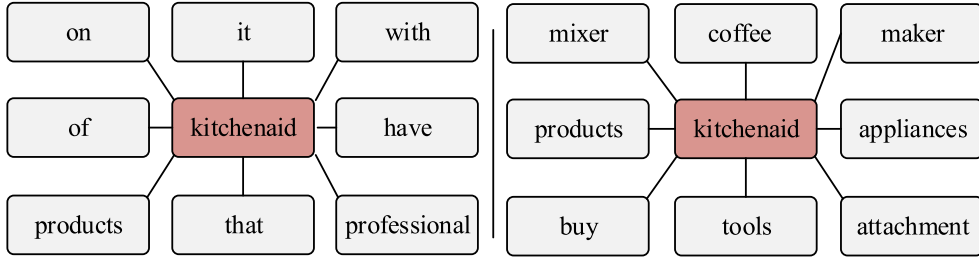


Fig. 2. Top related words of *KitchenAid*, captured by the co-occurrence matrix $\mathbf{M}$ (left), and the domain-specific graph $\mathcal{G}$ (right). As seen, $\mathbf{M}$ and $\mathcal{G}$ address different levels of semantic context/structures. A comprehensive use of both leads to a more complete understanding of the meaning of words.

where $\mathbf{u}_i \in \mathbb{R}^d$ is the final meta-embedding of w_i . To encourage the final meta-embedding to capture multi-order semantic dependency structures among domain-specific words, we improve Eq. (13) to:

$$\mathcal{L}_{\text{structure}} = - \sum_{k=0}^K \sum_{w_j \in \mathcal{V}} \left(\mathbf{u}_i^\top \mathbf{h}_i^k - \log \sum_{w_j \in \mathcal{V}} \exp(\mathbf{u}_j^\top \mathbf{h}_i^k) \right). \quad (14)$$

When $K = 0$, the final meta-embedding $\mathbf{u}_i$ is only required to be consistent with the initial input $\mathbf{v}_i$. When $K = 1$, the GCN encodes additional information from immediate neighbors into $\mathbf{u}_i$. When K is further increased (i.e., more GCN layers are stacked), $\mathbf{u}_i$ will encode K -order neighborhoods (i.e., information about nodes at most K hops away), but with more risk of contamination from irrelevant words. In our implementation, we set $K = 2$. The validation for this can be found in Section 5.

3.3. Learning process

We formulate the joint objective as a minimization problem as follows:

$$\mathcal{L} = \mathcal{L}_{\text{context}} + \lambda \mathcal{L}_{\text{structure}}, \quad (15)$$

where $\lambda \in \mathbb{R}^+$ is a non-negative real-valued combination hyper-parameter. To optimize the aforementioned model, we follow the GloVe method to minimize $\mathcal{L}_{\text{context}}$. We follow the GCN method [30] to optimize the $\mathcal{L}_{\text{structure}}$. During training, we use stochastic gradient descent (SGD), with learning rates scheduled by AdaGrad [38]. The overall algorithm for learning our meta-embeddings is listed in Algorithm 1.

Algorithm 1 Meta-Embedding Algorithm

Input: A set of source word embeddings $\{\mathbf{E}^n \in \mathbb{R}^{|\mathcal{V}_n| \times d_n}\}_{n=1}^N$, and the task domain corpus $\mathcal{C}$.

Parameter: Dimensionality d of word embeddings, the maximum number of iterations T .

Output: Each word's meta-embedding $\mathbf{u}_i \in \mathbb{R}^d$ in the task domain.

1: Initialize $\mathbf{c}$ and $\mathbf{x}$ for each word, and generate the word's context embedding $\tilde{\mathbf{x}}$.

2: Construct the co-occurrence matrix $\mathbf{M}$ and domain-specific graph $\mathcal{G}$

3: Project source embeddings into a common d -dimension space for each word w_i using Eq. (1)

4: **while** 1 to T

5: Optimize a , $\mathbf{c}$, and $\mathbf{x}$ according to the semantic structures with $\mathcal{L}$ in Eq. (15)

6: **end while**

7: **return** $\mathbf{u}_i$ of each word w_i

4. Experiments

In this section, we conduct comprehensive experiments to verify the effectiveness of our method. We first detail the implementation of the proposed domain-specific meta-embeddings, including the used source embeddings and the construction of the co-occurrence matrix and domain-specific graph. Then, we describe the used baseline methods. Following this procedure, we report the experiments on two tasks, i.e., text classification and relation extraction.

4.1. Implementation

Source Embeddings: Three widely-used pre-trained word embeddings are used as the source embeddings of our method:

- **CBOW**: Trained by continuous bag-of-words method and released by Mikolov et al. [1], CBOW has a vocabulary of 929,019 words. The dimension size is 300, the training corpus is Google News with 100 billion tokens.
- **GloVe**: The most widely used pre-trained word embeddings. GloVe is trained by Pennington et al. [2] using global word co-occurrences information in the corpus. There are 1,193,514 words in the vocabulary, and the dimension size is 300. The training corpus is web-crawled text with 42 billion tokens.
- **fastText**: Released by Mikolov et al. [18]. The pre-trained word embeddings of fastText are equipped with flexible sub-word structures. It is trained on the Wikipedia corpus with 16 billion tokens, with a vocabulary size of 1 million and dimensionality of 300.

In our experiments, the dimensionality of the learned meta-embeddings (d) is set to 300, and the evaluation of different dimensionality is detailed in Section 5. For the co-occurrence matrix $\mathbf{M}$, we set the context window to be the 10 tokens preceding and succeeding a given word in a sentence. We extract unigrams from the co-occurrence windows as the corresponding context words. We down-weight distant (and potentially noisy) co-occurrences using the reciprocal $\frac{1}{l}$, where l is the distance in tokens between two words that co-occur. When constructing the specific graph $\mathcal{G}$, we set each sentence as a window. In each window, stop-words are discarded and only the correlations among the top-10 most important words are counted (to reduce noise).

4.2. Baselines

For a comprehensive comparison, the proposed method is compared against state-of-the-art baselines from three classes:

- (1) Pre-trained source embeddings. Three state-of-the-art pre-trained word embeddings trained from large-scale corpora, namely CBOW, GloVe, and fastText, are compared in the experiments as detailed in Section 4.1.
- (2) Domain-specific embeddings. We train the domain-specific word embedding on the task domain corpus, using the Word2Vec and GloVe methods, denoted as CBOW^t , Skipgram^t , and GloVe^t , respectively. We use the official public tools with the default settings. The dimensionality is also set to 300.
- (3) Meta-embedding methods. We compare the proposed method with the four following state-of-the-art meta-embedding methods:
 - **1ToN** [10], which learns meta-embeddings for the words in an intersection vocabulary, by predicting their representations in the individual source embedding sets. 1ToN's extension, **1ToN+**, which is designed for handling OOVs, is also included in our experiments.
 - **LLE** [23], which generates the meta-embedding space by learning a locally-linear projection that predicts the words' neighborhoods in the source embeddings.
 - **AEME** [12] trains an auto-encoder to encode source embeddings into a common meta-embedding space and then decodes them back to the original source space.
 - **REGrep** [24] is a domain-specific word embedding method. It is trained on an in-domain corpus with a regularizer that constrains common words to be similar to their source embeddings.

We use the released embeddings of 1ToN and 1ToN+. LLE, AEME, and REGrep, which are trained using their codes published online with the same source embeddings as detailed in Section 4.1. Moreover, we train an extended version of these meta-embedding methods, denoted as $\text{LLE} + t$ and $\text{AEME} + t$, by integrating CBOW^t trained on the task domain as an extra source embedding.

4.3. Task I: Text Classification

The first evaluation task is conducted on text classification over three datasets:

- **Amazon Reviews** [39]. The dataset consists of product reviews in Amazon with labels to classify the review as positive or negative. Its four subsets are conducted in our experiments, i.e., **Kitchen**, **DVD**, **Electronics**, and **Book**.
- **Custom Reviews (CR)** [40]. The dataset contains various products (such as cameras) with labels to classify the review as positive or negative.
- **TREC** [41]. This is a question-type classification dataset, which coarsely classifies the question sentences into six types.

All the datasets are organized following their standard splits. The statistics of the used datasets are listed in Table 2. To conduct the experiment, two types of classifiers are used in our experiments:

- (1) **SVM**: Each document is represented as a bag-of-words, and we compute the centroid of the embeddings for each bag to represent the document. Then, we train a Linear SVM classifier using the training portion of each dataset and evaluate the classification accuracy using the corresponding testing portion.

Table 2

Statistics of the corpora used in our experiments. #Vocab is the size of vocabulary. Avg.Len. is the average length of documents in the corpus.

Dataset	#Train	#Test	#Vocab.	#Tokens	Avg.Len.	Size
Kitchen	1,600	400	9,066	140 K	69.33	711 KB
DVD	1,600	400	16,018	172 K	85.33	903 KB
Electronics	1,600	400	9,641	136 K	67.27	697 KB
Book	1,600	400	16,575	176 K	87.07	951 KB
CR	3,450	320	5,319	74 K	17.70	353 KB
TREC	5,452	500	8,604	54 K	9.71	264 KB

- (2) **CNN**: A CNN based classifier is trained with 100 filters, each of size 5, for max-pooling. The initial learning rate is set to 0.01, and the batch size is set to 50. The accuracy score averaged over ten repetitions is reported.

Table 3 lists the overall performances of the text classification task. The results show that our method significantly outperforms all other competitors across different settings. The improvement of our proposed meta-embedding method is statistically significant over the best single source embeddings, indicating the effectiveness of the technique. More specifically, the domain-specific embeddings, i.e., CBOW^t and GloVe^t, achieve poor performances compared with the source embeddings, especially on the Amazon Reviews dataset with only 2,000 reviews. The main reason for the poor performance is that the task domain corpus is too small to train reliable word representations, which suggests the importance of using extra knowledge from source embeddings to enrich the semantics of in-domain words.

Table 3 demonstrates that our method outperforms the other meta-embedding methods, indicating the advantage of our domain-specific meta-embeddings. Furthermore, our method achieves significant improvements over the meta-embeddings, which do not consider information in the task domain, such as 1ToN, 1ToN+, LLE and AEME. Both LLE + t and AEME + t outperform their basic methods, LLE and AEME, by 1.9% and 0.56%, respectively, indicating the significance of task domain information. These observations demonstrate the importance of adapting the learned embedding to the specific domain. Notably, one reason for the poor performance of LLE and AEME is that they suffer from 16.9% OOVs on average, for all the datasets. Our method alleviates this problem by introducing a learnable vector $\mathbf{c}$, thus achieves better performances. Our method achieves substantial gains compared with meta-embeddings which contain domain-specific information, i.e., LLE + t, AEME + t, and REGrep. For example, compared with the second-best method REGrep, our method surpasses it by 4.8% on average. This indicates that our proposed approach is more effective in the exploration of domain-specific knowledge.

In **Table 4**, we compare our method with classifiers build on pre-trained language models, i.e., BERT-base [3] (the uncased version) and RoBERTa-base [42]. In our experiment, the sequence length is set to 128, the learning rate is searched in range of {1e-5, 2e-5, 3e-5}, the training epoch is set to 3, and the batch size is set to 32. We observe that the pre-trained language models achieve better results. The main reasons are twofold: (1) the pre-trained language models are contextual representation method which can cover the long-term contextual information; (2) the pre-trained language models are trained on large-scale textual data using huge-amount computing resources, which convey rich linguistic knowledge and background knowledge. While our method is a distributed representation method, which is a lightweight and easy to be implemented approach. As such, it is unfair to compare our method with pre-trained models directly. Compared with other distributed methods, our method achieves better results and highlights the importance of domain-specific knowledge, which is also informative to improve the contextual representation methods.

4.4. Task II: Relation Extraction

We evaluate the proposed method using the relation extraction task on the New York Times corpus (NYT) dataset², developed by Riedel et al. [43]. This dataset was generated by aligning Freebase relations with the New York Times corpus. Entity mentions are found using the Stanford named entity tagger [44] and are further matched to the names of Freebase entities. The Freebase relations are divided into two parts, one for training and one for testing. To be specific, sentences from the years 2005–2006 are combined to form the training set, while sentences from 2007 are used for testing. There are 53 possible relationships, including a special relation NA, which indicates there is no relation between head and tail entities. The training data contains 522,611 sentences, 281,270 entity pairs and 18,252 relational facts. The testing set contains 172,448 sentences, 96,678 entity pairs and 1,950 relational facts. We use Piecewise Convolution Neural Networks (PCNNs) [45] as the distant supervised relation extraction model. In the experiments, different word representations are used to initialize the model. Precision@100, 200, and 300 (P@N) are reported.

The overall evaluation results are reported in **Table 5**. It is shown that, for the relation extraction task, our method also achieves a promising performance, with an average improvement of 6.4% over source embeddings and 5.8% over the meta-embeddings methods. The pre-trained source embeddings achieve better performance than domain-specific embeddings which are only trained on the task domain corpus, i.e., pre-trained GloVe beats GloVe^t by 2% in terms of P@100. This obser-

² <http://iesl.cs.umass.edu/riedel/ecml/>

Table 3

Overall performance for the text classification task, conducted on four subsets (i.e., *Kitchen*, *DVD*, *Book*, and *Electronics*) of *Amazon Reviews*, *Custom Reviews* (CR), and *TREC* datasets. The highest scores are marked in bold.

Methods	SVM classifier						CNN classifier					
	Amazon Reviews				CR	TREC	Amazon Reviews				CR	TREC
	Kitchen	DVD	Book	Elec.			Kitchen	DVD	Book	Elec.		
CBOW	82.00	79.00	81.50	83.75	81.56	78.20	81.53	69.25	69.63	77.43	82.72	85.28
GloVe	82.75	79.75	81.25	81.00	82.81	82.20	85.90	78.68	78.68	81.25	84.63	87.02
fastText	83.25	80.75	80.00	81.75	81.87	81.80	79.43	67.38	68.58	75.28	80.50	84.90
CBOW ^t	81.25	73.50	74.50	76.50	80.94	89.40	85.08	80.28	79.38	82.68	78.44	89.58
Skipgram ^t	82.00	73.75	73.75	76.75	80.25	89.64	85.88	80.65	79.03	82.90	78.63	89.88
GloVe ^t	78.50	68.00	68.75	71.25	73.13	70.20	83.85	75.10	73.43	79.58	75.97	83.78
1ToN	76.25	72.25	77.25	77.50	81.50	81.70	84.80	78.25	79.15	83.73	76.56	82.98
1ToN+	73.00	71.25	78.25	73.50	68.50	77.20	84.85	79.75	80.30	84.25	76.81	80.84
LLE	76.50	71.25	75.00	73.25	82.50	81.70	71.95	77.03	75.23	72.93	73.13	72.60
LLE + t	77.75	73.25	78.50	74.75	83.50	68.00	74.60	76.05	74.40	74.43	76.13	79.30
AEME	79.50	75.50	79.50	76.25	84.20	77.80	79.09	76.09	78.41	71.38	78.53	81.83
AEME + t	82.00	80.00	78.75	78.25	82.50	76.80	78.68	75.53	77.54	73.98	78.78	82.02
REGrep	81.00	77.80	77.80	78.00	75.60	87.80	82.15	79.23	78.80	79.38	83.28	90.20
Our method	84.75	80.75	84.00	84.50	84.69	92.30	88.15	82.30	82.50	85.85	86.59	92.78

Table 4

The comparison of our method with the pre-trained language models.

Methods	Amazon Reviews				CR	TREC
	Kitchen	DVD	Book	Elec.		
Our Methods-SVM	84.75	80.75	84.00	84.50	84.69	92.30
Our Methods-CNN	88.15	82.30	82.50	85.85	86.59	92.78
BERT-base	92.50	90.25	87.75	90.25	93.75	96.20
RoBERTa-base	93.50	91.00	92.25	92.50	95.63	96.80

Table 5

Overall performance for the relation extraction task. The best scores are marked in bold.

Methods	P@100	P@200	P@300
CBOW	78.00	73.50	66.00
GloVe	80.00	68.00	64.33
fastText	81.00	72.00	67.00
CBOW ^t	77.00	74.50	67.33
Skipgram ^t	78.00	73.50	67.37
GloVe ^t	78.00	70.00	63.33
1ToN	78.00	76.00	68.33
1ToN+	75.00	71.00	66.67
LLE	76.00	70.50	66.00
LLE + t	81.00	72.00	69.33
AEME	78.00	73.50	66.00
AEME + t	82.00	72.50	65.33
Our method	86.00	77.50	72.30

vation shows that the pre-trained source embeddings are useful and efficient resources for capturing word's semantics. The methods LLE + t and AEME + t achieve better results than their source embeddings, as well as their corresponding methods, LLE and AEME. These results suggest that task-domain information might be useful for guiding the integration of source embeddings. Our method jointly considers both the background knowledge from source embeddings and the domain-specific knowledge from the task domain. Moreover, our method consistently outperforms LLE + t and AEME + t, which also considers the domain-specific information. This performance is attribute to our GCN-based framework, which can capture more intricate correlations among domain-specific words, showing the superiority and generality of our technique for modeling domain-specific information in the word embedding learning process.

5. In-Depth Analyses and Case Study

In this section, we qualitatively and quantitatively investigate the effects of essential components in the proposed method, assess the impact of key parameters, and visualize the attentions. The experiments are conducted on the Amazon Reviews dataset.

5.1. Ablation study

Here, we assess the contribution of each component of the proposed method. One by one, each component is removed, and the test accuracy on the Amazon Review dataset. The SVM classifier is used for comparison. As shown in Table 6, our full meta-embedding method performs better than other variants when only task domain context or semantic structures are considered. This observation indicates that the domain-specific information is useful to improve the quality of meta-embeddings, either the contextual information from the raw corpus or the latent semantic structures conveyed by the graph of salient words. In our method, to better capture a word's context features, we explore background knowledge ($\{\mathbf{s}_i^n\}_{n=1}^N$ from source embeddings (Eq. (2)). To validate the effectiveness of source embeddings, we provide a baseline, *w/o source embeddings*, which does not use pre-trained embeddings. A significant drop in performance is observed, indicating that the source embeddings indeed provide useful background knowledge for capturing word's semantics in the embedding space.

Moreover, we verify the effectiveness of GCNs for capturing the latent semantic structures of the task domain. In our method, we capture multi-order relations among domain-specific words by stacking K GCN layers. To evaluate the influence of the GCN design, we report the performance as functions of a variety of K s. As shown in Table 6, the performance increases when more semantic structures are explored ($K \uparrow$). However, when more than three GCN layers are stacked, the performance becomes worse, due to disturbance from irrelevant words.

5.2. Case study

For an in-depth analysis, we conduct a case study to evaluate the proposed method qualitatively. To be specific, we consider the word *season* in the kitchen domain and show its predicted nearest neighbors in different embedding spaces. As shown in Table 7, the pre-trained source embedding (i.e., fastText) and meta-embeddings (i.e., LLE and AEME) predict the most closely related words as being *year*, *summer*, *holiday*, *week*. They are prone to capture the sense of *season* as one of the four periods of the year, which is the main semantics used in the general domain. However, the task domain is the kitchen domain. The meaning of *season* is to heighten or improve the flavor of food by adding condiments, spices, herbs, or the like. Being well-deserving of the domain-specific graph which provides more hints, our method reveals its domain-specific semantics. As shown in the table, our method is able to capture its strong relationship with *spices*, *sauce*, and *flavor*.

5.3. Parameter validation

We investigate the influence of the dimensionality d of the meta-embeddings and the impact of parameter α in Eq. (7). The performance with different values of d (i.e., $d \in \{50, 100, 200, 300, 500\}$) is shown in Fig. 3. We see a gradual improvement in performance with increasing d , which plateaus after $d = 300$, which indicates that adding new dimensions over 300 does not result in more performance improvement. Next, we report the performance for $\alpha = \{0.001, 0.01, 0.1, 1, 10, 20, 50\}$. As shown in the right of Fig. 3, $\alpha = 1$ achieves the best performance. The recommended settings for d and α is 300 and 1, respectively.

Table 6
Evaluation of essential components and different GCN designs. of the proposed method.

Method	Amazon Reviews			
	Kitchen	DVD	Book	Elec.
Full model ($K = 2$)	84.75	80.75	84.00	84.50
w/o task domain semantic structures	82.25	78.75	81.75	81.25
w/o task domain contextual information ($K = 2$)	74.25	73.75	72.50	74.50
w/o source embeddings ($K = 2$)	76.50	77.75	77.50	78.00
Full model ($K = 0$)	83.25	79.75	82.00	83.25
Full model ($K = 1$)	83.50	80.50	83.50	83.75
Full model ($K = 3$)	83.00	80.25	83.75	83.25
Full model ($K = 4$)	82.75	78.50	81.50	83.00

Table 7
Top five words related to *season* in different embedding spaces.

fastText	CBOW [†]	LLE	AEME	Our method (w/o semantic structures)	Our method (w semantic structures)
year	remembering	week	finale	all-clad	spices
finale	specifications	weekend	year	starbucks	sauce
off-season	record	finals	league	unpredictable	flavor
winter	holiday	playoffs	postseason	stones	ingredients
summer	load	doubleheader	playoffs	gourmet	marinade

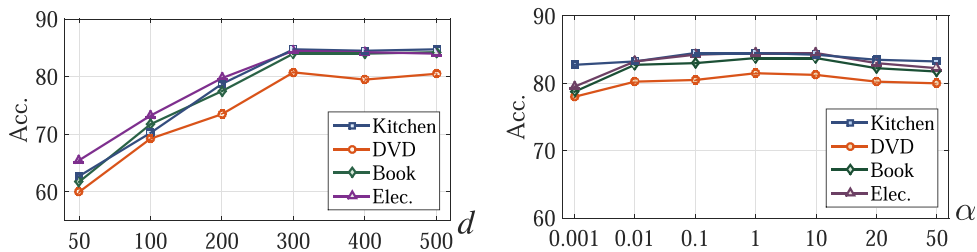


Fig. 3. Performance for parameter selection on text classification task for meta-embedding dimension d and α .

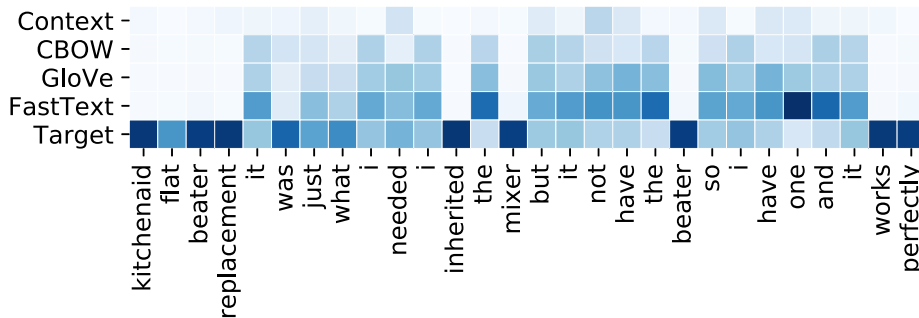


Fig. 4. Visualization of self-attention-based combination weights (a deeper color signifies a higher weight). CBOW, GloVe, and fastText are the pre-trained source embeddings. Context denotes $\mathbf{c}$. Target denotes $\mathbf{x}$.

5.4. Weight visualization

To gain an intuitive understanding of what happens when source embeddings are dynamically combined (Eq. (3)), we visualize the attention-based combination weights (a defined in Eq. (4)). As shown in Fig. 4, general words, such as *needed* and *have*, receive evenly divided attention across different embeddings. The method fastText is given relatively higher weights, possibly because it is trained on a more recent version of Wikipedia, and it is more favored for text classification. In contrast, domain-specific words, such as *KitchenAid*, *beater* and *mixer*, are more reliant on the target embedding $\mathbf{x}$ (denoted as *Target* in the last row), which conveys more structural information in the task domain.

6. Conclusions and further study

In this paper, we proposed an unsupervised meta-embedding method, that dynamically integrates pre-trained source embeddings to adapt the task domain by exploring background knowledge from source domains and domain-specific knowledge mining from both the word distribution in the corpus and correlations among significant words. The domain-specific words and their co-occurring information is used to construct the graph of the domain structure, and the GCN-based model is designed to explicitly preserve such structure consistency in the learned embedding space. Experiments on different tasks and in-depth analyses show that our method is able to produce accurate meta-embeddings of words, which efficiently address domain-specific knowledge.

In the future, there are two potential research directions. On the one hand, we will extend our method to multi-lingual word embedding construction. With a GCN-based domain-structure module, we will design a common embedding space by aligning domain-invariant words from different languages, as well as preserving domain-specific semantic structures in their individual language. On the other hand, in this paper, we reveal the importance of semantic structures and design an effective method to incorporate semantic structures and background knowledge conveyed by distributed word representations (i.e., source embeddings). Recently, pre-trained language models (e.g., BERT) have been shown to be more effective than the distributed word representations. In the future, we will explore how to integrate semantic structures into pre-trained language models.

CRediT authorship contribution statement

Qian Liu: Conceptualization, Methodology, Data curation, Writing - original draft. **Jie Lu:** Methodology, Writing - review & editing, Supervision. **Guangquan Zhang:** Data curation, Supervision. **Tao Shen:** Methodology. **Heyan Huang:** Supervision.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The work presented in this paper was supported by the Australian Research Council under Discovery Grant DP170101632 and the Joint Funds of the National Natural Science Foundation of China under Grant No.U19B2020.

References

- [1] Tomas Mikolov, Ilya Sutskever, Kai Chen, Gregory S. Corrado, Jeffrey Dean, Distributed representations of words and phrases and their compositionality, in: *Proceedings of NIPS*, 2013, pp. 3111–3119.
- [2] Jeffrey Pennington, Richard Socher, Christopher D. Manning, Glove: Global vectors for word representation, in: *Proceedings of EMNLP*, 2014, pp. 1532–1543.
- [3] Jacob Devlin, Ming-Wei Chang, Kenton Lee, B.E.R.T. Kristina Toutanova, pre-training of deep bidirectional transformers for language understanding, in: *Proceedings of NAACL-HLT*, 2019, pp. 4171–4186.
- [4] Zhilin Yang, Zihang Dai, Yiming Yang, Jaime G. Carbonell, Ruslan Salakhutdinov, and Quoc V. Le. Xlnet: Generalized autoregressive pretraining for language understanding. CoRR, abs/1906.08237, 2019.
- [5] Qian Liu, Heyan Huang, Yang Gao, Xiaochi Wei, Yuxin Tian, and Luyang Liu. Task-oriented word embedding for text classification, in: *Proceedings of COLING*, pp 2023–2032, 2018.
- [6] Chao Wang, Hui Jiang, Explicit utilization of general knowledge in machine reading comprehension, in: *Proceedings of ACL*, 2019, pp. 2263–2272.
- [7] Yanqing Chen, Bryan Perozzi, Rami Al-Rfou', Steven Skiena, The expressive power of word embeddings, *Proc. of ICML Workshop* (2013).
- [8] Siwei Lai, Kang Liu, Shizhu He, Jun Zhao, How to generate a good word embedding, *IEEE Intelligent Systems* 31 (6) (2016) 5–14.
- [9] Yuta Tsuboi, Neural networks leverage corpus-wide information for part-of-speech tagging, in: *Proceedings of EMNLP*, 2014, pp. 938–950.
- [10] Wenpeng Yin, Hinrich Schütze. Learning word meta-embeddings, in: *Proc. of ACL*, pages 1351–1360, 2016.
- [11] Joshua Coates, Danushka Bollegala, Frustratingly easy meta-embedding - computing meta-embeddings by averaging source word embeddings, in: *Proceedings of NAACL-HLT*, 2018, pp. 194–198.
- [12] Cong Bao, Danushka Bollegala. Learning word meta-embeddings by autoencoding, in: *Proc. of COLING*, pages 1650–1661, 2018.
- [13] Karen Spärck Jones, A statistical interpretation of term specificity and its application in retrieval, *J. Documentation* 60 (5) (2004) 493–502.
- [14] Yoshua Bengio, Réjean Ducharme, Pascal Vincent, Christian Janvin, A neural probabilistic language model, *J. Mach. Learn. Res.* 3 (2003) 1137–1155.
- [15] Bing Ma, Qi Qi, Jianxin Liao, Haifeng Sun, Jingyu Wang, Learning chinese word embeddings from character structural information, *Computer Speech Language* 60 (2020).
- [16] Danushka Bollegala, Yuichi Yoshida, Ken-ichi Kawarabayashi, Using k-way co-occurrences for learning word embeddings, in: *Proceedings of AAAI*, 2018, pp. 5037–5044.
- [17] Muhammad Atif Qureshi, Derek Greene, EVE: explainable vector based embedding technique using wikipedia, *J. Intell. Inform. Syst.* 53 (1) (2019) 137–165, <https://doi.org/10.1007/s10844-018-0511-x>.
- [18] Tomas Mikolov, Edouard Grave, Piotr Bojanowski, Christian Puhrsch, Armand Joulin. Advances in pre-training distributed word representations, in: *Proc. of LREC*, 2018.
- [19] Xipeng Qiu, Tianxiang Sun, Yige Xu, Yunfan Shao, Ning Dai, and Xuanjing Huang. Pre-trained models for natural language processing: A survey. CoRR, abs/2003.08271, 2020.
- [20] Christoph Alt, Marc Hübner, Leonhard Hennig, Fine-tuning pre-trained transformer language models to distantly supervised relation extraction, in: *Proceedings of ACL Association for Computational Linguistics*, 2019, pp. 1388–1398.
- [21] An Yang, Quan Wang, Jing Liu, Kai Liu, Yajuan Lyu, Hua Wu, Qiaoqiao She, Sujian Li. Enhancing pre-trained language representations with rich knowledge for machine reading comprehension, in: *Proceedings of ACL*, pp. 2346–2357, 2019.
- [22] Lu, Jie, Wahid Behbood, Peng Hao, Hu.a. Zuo, Shan Xue, Guangquan Zhang, Transfer learning using computational intelligence: A survey, *Knowledge Based Systems* 80 (2015) 14–23.
- [23] Danushka Bollegala, Kohei Hayashi, Ken-ichi Kawarabayashi, Think globally, embed locally - locally linear meta-embedding of words, in: *Proceedings of IJCAI*, 2018, pp. 3970–3976.
- [24] Wei Yang, Lu, Wei, Vincent Zheng, A simple regularization-based algorithm for learning cross-domain word embeddings, in: *Proceedings of EMNLP*, 2017, pp. 2898–2904.
- [25] Lu, Jie, Hu.a. Zuo, Guangquan Zhang, Fuzzy multiple-source transfer learning, *IEEE Trans. Fuzzy Systems* (2019).
- [26] Douwe Kiela, Changhan Wang, Kyunghyun Cho, Dynamic meta-embeddings for improved sentence representations, in: *Proceedings of EMNLP*, 2018, pp. 1466–1477.
- [27] Xu, Hu, Bing Liu, Lei Shu, S.Yu. Philip, Lifelong domain word embedding via meta-learning, in: *Proceedings of IJCAI*, 2018, pp. 4510–4516.
- [28] Amir Hazem, Emmanuel Morin, Leveraging meta-embeddings for bilingual lexicon extraction from specialized comparable corpora, in: *Proceedings of COLING*, 2018, pp. 937–949.
- [29] David K Duvenaud, Dougal Maclaurin, Jorge Iparraguirre, Rafael Bombarell, Timothy Hirzel, Alán Aspuru-Guzik, Ryan P Adams. Convolutional networks on graphs for learning molecular fingerprints, in: *Proc. of NIPS*, pp. 2224–2232, 2015.
- [30] Thomas N. Kipf, Max Welling, Semi-supervised classification with graph convolutional networks, in: *Proceedings of ICLR*, 2017.
- [31] Wenguan Wang, Lu. Xiankai, Jianbing Shen, David J. Crandall, Ling Shao, Zero-shot video object segmentation via attentive graph neural networks, in: *Proceedings of ICCV. IEEE*, 2019, pp. 9235–9244.
- [32] Siyuan Qi, Wenguan Wang, Baoxiong Jia, Jianbing Shen, Song-Chun Zhu. Learning human-object interactions by graph parsing neural networks, in: *Proc. of ECCV*, 2018.
- [33] Lu, Jie, Junyu Xuan, Guangquan Zhang, Xiangfeng Luo, Structural property-aware multilayer network embedding for latent factor analysis, *Pattern Recognition* 76 (2018) 228–241.
- [34] Joost Bastings, Ivan Titov, Wilker Aziz, Diego Marcheggiani, Khalil Sima'an. Graph convolutional encoders for syntax-aware neural machine translation, in: *Proc. of EMNLP*, pp. 1957–1967, 2017.
- [35] Thien Huu Nguyen, Ralph Grishman. Graph convolutional networks with argument-aware pooling for event detection, in: *Proc. of AAAI*, pp. 5900–5907, 2018.
- [36] Qian Liu, Heyan Huang, Guangquan Zhang, Yang Gao, Junyu Xuan, Jie Lu. Semantic structure-based word embedding by incorporating concept convergence and word divergence, in: *Proceedings of AAAI*, pp. 5261–5268, 2018.
- [37] Hien T. Nguyen, Phuc H. Duong, and Erik Cambria. Learning short-text semantic similarity with word embeddings and external knowledge sources. *Knowledge Based Systems*, 182, 2019.

- [38] John C. Duchi, Elad Hazan, Yoram Singer, Adaptive subgradient methods for online learning and stochastic optimization, *J. Mach. Learning Res.* 12 (2011) 2121–2159.
- [39] John Blitzer, Mark Dredze, Fernando Pereira, Biographies, bollywood, boom-boxes and blenders: Domain adaptation for sentiment classification, in: *Proceedings of ACL*, 2007.
- [40] Minqing Hu, Bing Liu. Mining and summarizing customer reviews, in: *Proceedings of SIGKDD*, pp. 168–177, 2004.
- [41] Xin Li, Dan Roth, Learning question classifiers, in: *Proceedings of COLING*, 2002.
- [42] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, Veselin Stoyanov. Roberta: A robustly optimized BERT pretraining approach. *CoRR*, abs/1907.11692, 2019.
- [43] Sebastian Riedel, Limin Yao, Andrew McCallum. Modeling relations and their mentions without labeled text, in: *Proc. of ECML PKDD*, pp. 148–163, 2010.
- [44] Jenny Rose Finkel, Trond Grenager, Christopher D. Manning. Incorporating non-local information into information extraction systems by gibbs sampling, in: *Proc. of ACL*, pp. 363–370, 2005.
- [45] Daojian Zeng, Kang Liu, Yubo Chen, Jun Zhao. Distant supervision for relation extraction via piecewise convolutional neural networks, in: *Proc. of EMNLP*, pp. 1753–1762, 2015.